

Caso de Uso 1: Registrar y Analizar Datos Climáticos

Actores:

- Sistema (automático)
- Administrador (opcional, para configurar o revisar)

Descripción:

El sistema obtiene datos de temperatura y humedad desde una API de pronóstico (MockForecastAPI), los registra en un repositorio, los analiza y, si detecta anomalías, envía una alerta por correo.

Flujo Principal:

1. El DatacenterWeatherManager invoca al IForecastAPI para obtener datos simulados.
2. Los datos (temperatura y humedad) son asignados a las instancias de TemperatureControl y HumidityControl.
3. Se crea un WeatherRecord con los valores y la fecha actual.
4. El WeatherRecord se guarda en el DataRepository.
5. El DatacenterWeatherManager ejecuta analyzeData().
6. Si se detecta una condición anómala (por ejemplo, temperatura > 30°C o humedad < 20% o > 80%), se llama a sendAlert().

Flujo Alternativo:

Si la API falla, el sistema registra un error pero no envía alerta (puede ser manejado por log o notificación interna).

Postcondición:

Los datos están almacenados y se ha evaluado su estado. Si hay anomalía, se envió alerta.

Caso de Uso 2: Consultar Último Registro Climático

Actores:

- Administrador del Datacenter
- Sistema de reportes

Descripción:

Un usuario o sistema externo solicita el último registro climático almacenado para visualización o toma de decisiones.

Flujo Principal:

1. El DatacenterWeatherManager recibe una solicitud de consulta.
2. Invoca al DataRepository para obtener el último registro (getLastRecord()).
3. Devuelve el WeatherRecord con temperatura, humedad y fecha.

Flujo Alternativo:

Si no hay registros, devuelve null o un mensaje “No hay datos disponibles”.

Postcondición:

Se entrega el último registro climático disponible.

Diagrama de secuencia – secuencia de mensajes

[DatacenterWeatherManager] → [MockForecastAPI]: sendTemperature(value)

[MockForecastAPI] → [DatacenterWeatherManager]: (retorno implícito, valor asignado)

[DatacenterWeatherManager] → [TemperatureControl]: increase(value) / decrease(value)
→ ajusta temperatura

[DatacenterWeatherManager] → [MockForecastAPI]: sendHumidity(value)

[MockForecastAPI] → [DatacenterWeatherManager]: (retorno implícito)

[DatacenterWeatherManager] → [HumidityControl]: increase(value) / decrease(value) →
ajusta humedad

[DatacenterWeatherManager] → [TemperatureControl]: read()

[DatacenterWeatherManager] → [HumidityControl]: read()

[DatacenterWeatherManager] → [DataRepository]: addRecord(WeatherRecord{temp, hum, date})

[DatacenterWeatherManager] → [DataRepository]: getLastRecord() → (para análisis, opcional)

[DatacenterWeatherManager] → [DatacenterWeatherManager]: analyzeData()

→ SI SE DETECTA ANOMALÍA:

[DatacenterWeatherManager] → [MailService]: sendAlert("Temperatura crítica: X°C")

Relaciones diagrama de clases UML

- IClimateVariable: Interfaz para controlar variables climáticas (temperatura/humedad). Define operaciones comunes.
- TemperatureControl / HumidityControl: Implementan la interfaz. Manejan valores numéricos y permiten leer/modificar.
- DatacenterWeatherManager: Clase principal. Coordina todo: obtiene datos, registra, analiza, alerta.
- DataRepository: Almacena registros históricos de clima.
- WeatherRecord: Estructura de datos que contiene temperatura, humedad y fecha.
- IForecastAPI: Interfaz para obtener datos climáticos externos.
- MockForecastAPI: Implementación simulada de la API (para pruebas).
- MailService: Envía alertas por correo cuando hay anomalías.